

装订线内

不要答题

北京大学信息科学技术学院考试试卷

考试科目：计算机系统导论 姓名：\_\_\_\_\_ 学号：\_\_\_\_\_  
考试时间：2025 年 12 月 15 日 大班号：\_\_\_\_\_ 小班号：\_\_\_\_\_

|     |    |    |    |    |    |    |    |    |    |    |
|-----|----|----|----|----|----|----|----|----|----|----|
| 题号  | L0 | L1 | L2 | L3 | L4 | L5 | L6 | L7 | L8 | 总分 |
| 分数  |    |    |    |    |    |    |    |    |    |    |
| 阅卷人 |    |    |    |    |    |    |    |    |    |    |

北京大学考场纪律

- 1、考生进入考场后，按照监考老师安排隔位就座，将学生证放在桌面上。无学生证者不能参加考试；迟到超过 15 分钟不得入场。在考试开始 30 分钟后方可交卷出场。
- 2、除必要的文具和主考教师允许的工具书、参考书、计算器以外，其它所有物品（包括空白纸张、手机、或有存储、编程、查询功能的电子用品等）不得带入座位，已经带入考场的必须放在监考人员指定的位置。
- 3、考试使用的试题、答卷、草稿纸由监考人员统一发放，考试结束时收回，一律不准带出考场。若有试题印制问题请向监考教师提出，不得向其他考生询问。提前答完试卷，应举手示意请监考人员收卷后方可离开；交卷后不得在考场内逗留或在附近高声交谈。未交卷擅自离开考场，不得重新进入考场答卷。考试结束时间到，考生立即停止答卷，在座位上等待监考人员收卷清点后，方可离场。
- 4、考生要严格遵守考场规则，在规定时间内独立完成答卷。不准交头接耳，不准偷看、夹带、抄袭或者有意让他人抄袭答题内容，不准接传答案或者试卷等。凡有违纪作弊者，一经发现，当场取消其考试资格，并根据《北京大学本科考试工作与学术规范条例》及相关规定严肃处理。
- 5、考生须确认自己填写的个人信息真实、准确，并承担信息填写错误带来的一切责任与后果。
- 学校倡议所有考生以北京大学学生的荣誉与诚信答卷，共同维护北京大学的学术声誉。

## 注意事项:

- (1) 本试卷基本按照实验发放次序分组, 不是按照题目难度排序。注意合理安排时间。
- (2) 除特别说明外, 默认技术前提是本课程教材内容与实验平台环境, 例如: x86-64 体系结构、Linux 操作系统、C 语言程序等; 汇编程序是对应 C 程序主体功能实现, 不做高级别优化, 不一定是完整代码。涉及 Y86-64 体系结构时, 采用课程中讲授的 SEQ 和 PIPE 等处理器结构。
- (3) 请在试卷上的每个实验标题后已画好的解答表格中答题, 其它位置视为放弃解答。

实验 10 (20 分) 得分: \_\_\_\_\_

|    |   |   |   |   |    |
|----|---|---|---|---|----|
| 题号 | 1 | 2 | 3 | 4 | 5  |
| 解答 |   |   |   |   |    |
| 题号 | 6 | 7 | 8 | 9 | 10 |
| 解答 |   |   |   |   |    |

1. (2 分) 以下哪个 `ls` 命令选项用于显示所有文件 (包括隐藏文件, 以 `.` 开头的文件)
  - A. `ls -l`
  - B. `ls -a`
  - C. `ls -h`
  - D. `ls -t`
2. (2 分) 使用 `cd` 命令时, 哪个参数表示 “上一次所在目录”
  - A. `~` (波浪符)
  - B. `...` (两个点)
  - C. `-` (减号)
  - D. `/` (斜杠)

3. (2分) 执行 `chmod 755 file.py` 后, 文件 `file.py` 的权限是:
- A. 所有者: 读写执行 (rwx), 组和其他: 读和执行 (r-x)
  - B. 所有者: 读写 (rw-), 组和其他: 读 (r--)
  - C. 所有者: 读和执行 (r-x), 组和其他: 读写 (rw-)
  - D. 所有者: 读写执行 (rwx), 组和其他: 读写 (rw-)
4. (2分) 下列关于 Linux 命令和命令行的使用, 说法错误的一项是:
- A. 在 Windows 中解压 tar 文件可能导致权限位丢失和文件名大小写问题, 因此最好在 Linux 中使用 tar 命令解压
  - B. gcc 命令可用于编译 C 语言代码, 其中 -o 选项用来指定输出的文件, -O 选项用来指定优化级别
  - C. rm 命令用于删除单个文件或多个文件, 如果要删除空目录只能用 rmdir 命令
  - D. grep 命令中, -i 选项的含义是“忽略大小写”, 如 `grep -i "hello" file.txt` 可匹配 Hello、HELLO 等
5. (2分) 使用 `ssh-keygen` 命令可以生成用于 SSH 登录的密钥。该命令通常生成两个文件: 公钥 `key.pub` 和私钥 `key`。这两个文件应存放在:
- A. 公钥留存在本地; 私钥留存在本地
  - B. 公钥留存在本地; 私钥上传到服务器
  - C. 公钥上传到服务器; 私钥留存在本地
  - D. 公钥上传到服务器; 私钥上传到服务器
6. (2分) 远程主机 10.0.0.5 的 SSH 服务端口为 2222 (非默认 22 端口), 若要将本地文件 `data.tar.gz` 复制到该主机 ubuntu 用户的 `~/backup/` 目录, 正确的 scp 命令是:
- A. `scp -p 2222 ubuntu@10.0.0.5:~/backup/ data.tar.gz`
  - B. `scp -P 2222 data.tar.gz ubuntu@10.0.0.5:~/backup/`
  - C. `scp ubuntu@10.0.0.5:2222:~/backup/ data.tar.gz`
  - D. `scp data.tar.gz ubuntu:2222@10.0.0.5:/backup/`

7. (2分) 在 Makefile 模式规则中, `%.o: %.c` 表示:
- A. 任一 .c 文件修改后, 所有的 .o 文件都需要重新编译
  - B. 每个 .o 文件依赖对应的同名 .c 文件
  - C. 当前目录中, 需要存在一个名为 `%.c` 文件, 否则 make 会报错
  - D. 当前目录中, 如果存在一个名为 `%.c` 文件, 会编译生成 `%.o` 文件
8. (2分) 关于 make 的执行过程, 以下说法正确的是:
- A. make 默认会执行 Makefile 中定义的所有规则
  - B. 如果没有指定目标, make 会执行第一个规则
  - C. make 会按照规则的书写顺序依次执行
  - D. Makefile 中规则的顺序是任意的, make 会自行决定执行顺序
9. (2分) 关于汇编和反汇编的说法。下列哪个选项是正确的?
- A. 反汇编得到的指令与原来的汇编文件的指令只需保证功能相同, 并非一一对应
  - B. 必须先将汇编代码转换为 C 源文件, 然后使用 gcc 编译得到目标文件, 最后使用 `objdump` 反汇编
  - C. 可以使用 `gcc -c` 命令将汇编代码直接生成目标文件, 然后使用 `objdump -d` 命令进行反汇编
  - D. 可以直接使用 `objdump` 直接从汇编代码生成指令的字节编码, gcc 在这个过程中没有作用
10. (2分) 在 binutils 中, 哪个工具可以用于查看共享库的依赖关系?
- A. `ldd`
  - B. `readelf`
  - C. `nm`
  - D. `objdump`

实验 L1 (10 分) 得分: \_\_\_\_\_

| 题号 | 11 | 12 | 13 | 14 | 15 |
|----|----|----|----|----|----|
| 解答 |    |    |    |    |    |

11. (2 分) 下面给出了 satMul2 函数的代码, 其中变量 overflow 在结果溢出时会被设置为 0xFFFFFFFF, 否则为 0。下面哪一个选项正确给出了 overflow 的表达式 (提示: 可根据 x 和 2 \* x 的符号位来判断是否溢出):

- A. (x ^ (x + 1)) >> 31;
- B. (x ^ (x + ~0)) >> 31;
- C. (x ^ (x << 1)) >> 31;
- D. (x ^ (x >> 1)) >> 31;

```
/*
 * satMul2 - multiplies by 2, saturating to Tmin or Tmax if overflow
 * Examples: satMul2(0x30000000) = 0x60000000
 *           satMul2(0x40000000) = 0x7FFFFFFF (saturate to Tmax)
 *           satMul2(0x90000000) = 0x80000000 (saturate to Tmin)
 */
int satMul2(int x) {
    int overflow, negative_x, raw_result, tmin, tmax;
    overflow = _____
    negative_x = x >> 31;
    raw_result = x << 1;
    tmin = 1 << 31;
    tmax = tmin + ~0;
    return (overflow & negative_x & tmin) | (overflow & ~negative_x &
tmax) | (~overflow & raw_result);
}
```

12. (2分) 下面给出了 bitParity 函数的代码, 该函数用于判断整数 x 的二进制表示中 0 的个数是否为奇数。设整数 x 的二进制表示为  $x[31]x[30]\dots x[1]x[0]$ , 则  $x1[15]$  的值可表示为:

- A.  $x[31] \wedge x[15]$
- B.  $x[31] \wedge x[16]$
- C.  $x[30] \wedge x[15]$
- D.  $x[30] \wedge x[16]$

```
/*
 * bitParity - returns 1 if x contains an odd number of 0's
 *   Examples: bitParity(5) = 0, bitParity(7) = 1
 */
int bitParity(int x) {
    int x1, x2, x3, x4, x5;
    x1 = x ^ (x >> 16);
    x2 = x1 ^ (x1 >> 8);
    x3 = x2 ^ (x2 >> 4);
    x4 = x3 ^ (x3 >> 2);
    x5 = x4 ^ (x4 >> 1);
    return x5 & 1;
}
```

13. (2分) 下面给出了 float\_half 函数的代码, 该函数给出单精度浮点数 f 的一半 (即  $0.5 * f$ ) 的位模式。下面哪一个选项正确给出了 (1) 和 (2) 处应当填写的值?

- A. (1) 0x800000 (2) 8
- B. (1) 0x800000 (2) 23
- C. (1) 0x400000 (2) 8
- D. (1) 0x400000 (2) 23

```
/*
 * float_half - Return bit-level equivalent of expression 0.5*f for
 *   floating point argument f.
 *   Both the argument and result are passed as unsigned int's, but
 *   they are to be interpreted as the bit-level representation of
 *   single-precision floating point values.
 *   When argument is NaN, return argument
 */
```

```

*/
unsigned float_half(unsigned uf) {
    unsigned sign = uf >> 31;
    unsigned exp = (uf >> 23) & 0xFF;
    unsigned frac = uf & 0x7FFFFFFF;
    /* Only roundup case will be when rounding to even */
    unsigned roundup = (frac & 0x3) == 3;
    if (exp == 0) {
        /* Denormalized. Must halve fraction */
        frac = (frac >> 1) + roundup;
    } else if (exp < 0xFF) {
        /* Normalized. Decrease exponent */
        exp--;
        if (exp == 0) {
            /* Denormalize adding back leading one */
            frac = (frac >> 1) + roundup + _____(1)_____;
        }
    }
    /* NaN Infinity do not require any changes */
    return (sign << 31) | (exp << _____(2)_____) | frac;
}

```

14. (2 分) 下面给出了 byteSwap 函数的代码，该函数交换后两个参数所指定的两个字节的数据。下面哪一个选项正确给出了 (1) 和 (2) 处应当填写的内容？

- A. (1) 3 (2) ^
- B. (1) 8 (2) ^
- C. (1) 3 (2) |
- D. (1) 8 (2) |

```

/*
 * byteSwap - swaps the nth byte and the mth byte
 * Examples: byteSwap(0x12345678, 1, 3) = 0x56341278
 *           byteSwap(0xDEADBEEF, 0, 2) = 0xDEEFBEAD
 * You may assume that 0 <= n <= 3, 0 <= m <= 3
 */

```

```

int byteSwap(int x, int n, int m) {
    int n8 = n << ____ (1) ____;
    int m8 = m << ____ (1) ____;

    int swap_byte = ((x >> m8) ____ (2) ____ (x >> n8)) & 0xff;

    return x ^ (swap_byte << n8) ____ (2) ____ (swap_byte << m8);
}

```

15. (2 分) 下面给出了 oddBits 函数的代码, 该函数将所有奇数位设置为 1 (设整数  $x$  的二进制表示为  $x[31]x[30]\dots x[1]x[0]$ , 奇数位是指  $j$  为奇数的所有  $x[j]$ )。

下面哪个选项给出了 byte 的正确设置:

- A. 0x55
- B. 0xAA
- C. 0x11
- D. 0xff

```

/*
 * oddBits - return word with all odd-numbered bits set to 1
 */
int oddBits(void) {
    int byte = ____;
    int word = byte | byte<<8;
    return word | word<<16;
}

```



## 实验 L2（10 分）得分：\_\_\_\_\_

| 题号 | 16 | 17 | 18 | 19 | 20 |
|----|----|----|----|----|----|
| 解答 |    |    |    |    |    |

16.（2 分）GDB 中断点功能的一种实现方式是：GDB 修改目标程序断点处的机器代码，让 CPU 执行到该位置时将控制权交给 GDB；之后 GDB 会恢复现场，包括将原指令写回、调整 `%rip` 回到断点指令处等，以使程序可以继续执行。基于这一机制，下列哪一项最符合实际情况？

- A. GDB 将断点处指令修改为 `NOP`，CPU 执行后不做任何操作，程序会暂停并等待
- B. GDB 将断点处指令修改为 `INT3`，CPU 执行后触发陷阱，陷入内核
- C. GDB 将断点处指令修改为 `sysenter`，CPU 执行后陷入内核
- D. GDB 将断点处指令修改为一条非法指令，类似于系统调用，CPU 执行后会陷入内核

17.（2 分）GDB 是一款功能强大的调试工具，下列关于 GDB 的用法错误的是？

- A. BombLab 中为了避免炸弹爆炸，可使用命令 `break explode_bomb` 下断点，使程序自动跳过 `explode_bomb` 函数的执行
- B. 命令 `info registers` 可显示程序中所有寄存器的值，包括 16 个通用寄存器和程序计数器、条件码寄存器等其他寄存器
- C. 命令 `ni`，`si` 都会执行下一条指令，但如果该指令是函数调用，命令 `ni` 不会进入函数内部，命令 `si` 会进入函数内部
- D. 命令 `p $rsp` 表示将 `$rsp` 寄存器中的值打印出来，如果 `p` 后面加上 `/x` 或 `/d`，表示打印成以十六进制或整数的形式

18.（2 分）下面是 `phase_1` 函数的反汇编代码，下面描述正确的是：

- A. `phase_1` 中需要比较两个字符串是否相等，即使输入的字符串更长，但只要内置字符串是输入字符串的前缀子串，就会判定为两个字符串相等
- B. `strings_not_equal` 是库函数，可以通过替换 `libc` 库使该函数返回 0，从而直接完成 `phase_1`
- C. 利用 `binutils` 的 `strings` 命令可以快速找到 `phase_1` 需要比对的内置字符串
- D. 只查看 `phase_1` 的反汇编和寄存器信息无法获得需要比对的内置字符串的地址

0000000000002798 <phase\_1>:

```
2798:  f3 0f 1e fa      endbr64
279c:  48 83 ec 08      sub    $0x8,%rsp
27a0:  48 8d 35 a1 1b 00 00 lea    0x1ba1(%rip),%rsi
                        # 4348 <_IO_stdin_used+0x348>
27a7:  e8 a6 05 00 00   call   2d52 <strings_not_equal>
27ac:  85 c0            test   %eax,%eax
27ae:  75 05            jne    27b5 <phase_1+0x1d>
27b0:  48 83 c4 08      add    $0x8,%rsp
27b4:  c3              ret
27b5:  e8 c7 06 00 00   call   2e81 <explode_bomb>
27ba:  eb f4            jmp     27b0 <phase_1+0x18>
```

19. (2 分) 下面是 phase\_2 函数的反汇编代码。下列哪个输入使 bomb 爆炸?

- A. 0 1 1 2 3 5
- B. 0 1 1 3 5 11
- C. 2 3 5 8 12 17
- D. 2 3 5 9 17 33

00000000000027bc <phase\_2>:

```
27bc:  f3 0f 1e fa      endbr64
27c0:  53              push   %rbx
27c1:  48 83 ec 20      sub    $0x20,%rsp
27c5:  64 48 8b 04 25 28 00 mov    %fs:0x28,%rax
27cc:  00 00
27ce:  48 89 44 24 18   mov    %rax,0x18(%rsp)
27d3:  31 c0            xor     %eax,%eax
27d5:  48 89 e6         mov    %rsp,%rsi
27d8:  e8 d0 06 00 00   call   2ead <read_six_numbers>
27dd:  83 3c 24 00      cmpl   $0x0, (%rsp)
27e1:  78 07            js      27ea <phase_2+0x2e>
27e3:  bb 01 00 00 00   mov    $0x1,%ebx
27e8:  eb 0a            jmp     27f4 <phase_2+0x38>
27ea:  e8 92 06 00 00   call   2e81 <explode_bomb>
```

```

27ef:  eb f2                jmp     27e3 <phase_2+0x27>
27f1:  83 c3 01             add     $0x1,%ebx
27f4:  83 fb 05             cmp     $0x5,%ebx
27f7:  7f 1c                jg      2815 <phase_2+0x59>
27f9:  48 63 c3             movslq  %ebx,%rax
27fc:  8d 53 ff             lea     -0x1(%rbx),%edx
27ff:  48 63 d2             movslq  %edx,%rdx
2802:  8b 14 94             mov     (%rsp,%rdx,4),%edx
2805:  8d 54 12 ff          lea     -0x1(%rdx,%rdx,1),%edx
2809:  39 14 84             cmp     %edx, (%rsp,%rax,4)
280c:  74 e3               je      27f1 <phase_2+0x35>
280e:  e8 6e 06 00 00      call    2e81 <explode_bomb>
2813:  eb dc               jmp     27f1 <phase_2+0x35>
2815:  48 8b 44 24 18      mov     0x18(%rsp),%rax
281a:  64 48 2b 04 25 28 00 sub     %fs:0x28,%rax
2821:  00 00
2823:  75 06               jne     282b <phase_2+0x6f>
2825:  48 83 c4 20          add     $0x20,%rsp
2829:  5b                  pop     %rbx
282a:  c3                  ret
282b:  e8 60 fa ff ff      call    2290 <__stack_chk_fail@plt>

```

20. (2分) 某同学在使用 GDB 调试 BombLab phase\_3 时, 在 \*phase\_3 + 0x84 处设置了断点。然后该同学在 phase\_3 中输入了 0 0, 触发了断点, 并在断点处输入了 info registers, 获得寄存器信息。下面是 phase\_3 的反汇编信息和断点处的寄存器信息。下列哪个 phase\_3 的输入不会使 bomb 爆炸?
- A. 0 -197
  - B. 1 -901
  - C. 6 0
  - D. 7 -772

```
# objdump -d bomb
```

```
00000000000001829 <phase_3>:
```

```
1829:  f3 0f 1e fa          endbr64
182d:  48 83 ec 18          sub    $0x18,%rsp
1831:  64 48 8b 04 25 28 00 mov    %fs:0x28,%rax
1838:  00 00
183a:  48 89 44 24 08      mov    %rax,0x8(%rsp)
183f:  31 c0              xor    %eax,%eax
1841:  48 8d 4c 24 04      lea    0x4(%rsp),%rcx
1846:  48 89 e2          mov    %rsp,%rdx
1849:  48 8d 35 50 2e 00 00 lea    0x2e50(%rip),%rsi
                                # 46a0 <transition_table+0x340>
1850:  e8 eb fa ff ff      call   1340 <__isoc99_sscanf@plt>
1855:  83 f8 01          cmp    $0x1,%eax
1858:  7e 1e          jle    1878 <phase_3+0x4f>
185a:  83 3c 24 07      cmpl   $0x7, (%rsp)
185e:  0f 87 9a 00 00 00   ja     18fe <phase_3+0xd5>
1864:  8b 04 24          mov    (%rsp),%eax
1867:  48 8d 15 b2 2a 00 00 lea    0x2ab2(%rip),%rdx
                                # 4320 <_IO_stdin_used+0x320>
186e:  48 63 04 82      movslq (%rdx,%rax,4),%rax
1872:  48 01 d0          add    %rdx,%rax
1875:  3e ff e0      notrack jmp  *%rax
1878:  e8 07 08 00 00      call   2084 <explode_bomb>
187d:  eb db          jmp    185a <phase_3+0x31>
187f:  b8 00 00 00 00      mov    $0x0,%eax
```

```

1884: 2d 1a 03 00 00      sub    $0x31a,%eax
1889: 05 99 02 00 00      add    $0x299,%eax
188e: 2d 04 03 00 00      sub    $0x304,%eax
1893: 05 04 03 00 00      add    $0x304,%eax
1898: 2d 04 03 00 00      sub    $0x304,%eax
189d: 05 04 03 00 00      add    $0x304,%eax
18a2: 2d 04 03 00 00      sub    $0x304,%eax
18a7: 83 3c 24 05          cmpl   $0x5,(%rsp)
18ab: 7f 06                jg     18b3 <phase_3+0x8a>
18ad: 39 44 24 04          cmp    %eax,0x4(%rsp)
18b1: 74 05                je     18b8 <phase_3+0x8f>
18b3: e8 cc 07 00 00      call  2084 <explode_bomb>
18b8: 48 8b 44 24 08      mov    0x8(%rsp),%rax
18bd: 64 48 2b 04 25 28 00 sub    %fs:0x28,%rax
18c4: 00 00
18c6: 75 42                jne    190a <phase_3+0xe1>
18c8: 48 83 c4 18          add    $0x18,%rsp
18cc: c3                  ret
18cd: b8 bf 02 00 00      mov    $0x2bf,%eax
18d2: eb b0                jmp    1884 <phase_3+0x5b>
18d4: b8 00 00 00 00      mov    $0x0,%eax
18d9: eb ae                jmp    1889 <phase_3+0x60>
18db: b8 00 00 00 00      mov    $0x0,%eax
18e0: eb ac                jmp    188e <phase_3+0x65>
18e2: b8 00 00 00 00      mov    $0x0,%eax
18e7: eb aa                jmp    1893 <phase_3+0x6a>
18e9: b8 00 00 00 00      mov    $0x0,%eax
18ee: eb a8                jmp    1898 <phase_3+0x6f>
18f0: b8 00 00 00 00      mov    $0x0,%eax
18f5: eb a6                jmp    189d <phase_3+0x74>
18f7: b8 00 00 00 00      mov    $0x0,%eax
18fc: eb a4                jmp    18a2 <phase_3+0x79>
18fe: e8 81 07 00 00      call  2084 <explode_bomb>
1903: b8 00 00 00 00      mov    $0x0,%eax
1908: eb 9d                jmp    18a7 <phase_3+0x7e>
190a: e8 91 f9 ff ff      call  12a0 <__stack_chk_fail@plt>

```

```
# objdump -s -j .rodata bomb
```

```
4320 add5ffff 5fd5ffff b4d5ffff bbd5ffff ....._.....
```

```
4330 c2d5ffff c9d5ffff d0d5ffff d7d5ffff ....._.....
```

```
(gdb) info registers
```

|         |                  |                              |
|---------|------------------|------------------------------|
| rax     | 0xffffffff3a     | 4294967098                   |
| rbx     | 0x55555555e6a0   | 93824992274080               |
| rcx     | 0x0              | 0                            |
| rdx     | 0x555555558320   | 93824992248608               |
| rsi     | 0x0              | 0                            |
| rdi     | 0x7fffffffffd3c0 | 140737488344000              |
| rbp     | 0x7fffffffffdae0 | 0x7fffffffffdae0             |
| rsp     | 0x7fffffffffda20 | 0x7fffffffffda20             |
| r8      | 0xffffffffff     | 4294967295                   |
| r9      | 0x0              | 0                            |
| r10     | 0x7ffff7db1fc0   | 140737351720896              |
| r11     | 0x7fffffffffd840 | 140737488345152              |
| r12     | 0x1              | 1                            |
| r13     | 0x0              | 0                            |
| r14     | 0x55555555acc8   | 93824992259272               |
| r15     | 0x7ffff7ffd000   | 140737354125312              |
| rip     | 0x5555555558ad   | 0x5555555558ad <phase_3+132> |
| eflags  | 0x293            | [ CF AF SF IF ]              |
| cs      | 0x33             | 51                           |
| ss      | 0x2b             | 43                           |
| ds      | 0x0              | 0                            |
| es      | 0x0              | 0                            |
| fs      | 0x0              | 0                            |
| gs      | 0x0              | 0                            |
| fs_base | 0x7ffff7fa1740   | 140737353750336              |
| gs_base | 0x0              | 0                            |

## 实验 L3（10 分）得分： \_\_\_\_\_

| 题号 | 21 | 22 | 23 | 24 | 25 |
|----|----|----|----|----|----|
| 解答 |    |    |    |    |    |

21.（2 分）在 AttackLab 中，hex2raw 工具用于将表示十六进制的字符串转换为字节序列，以便将攻击代码注入到目标程序中。假设现在有一个包含十六进制字符串的文件 hex.txt，下列哪个选项是错误的？

- A. 可以使用命令 `cat hex.txt | hex2raw > raw.bin` 将 hex.txt 中的内容转换为字节序列并保存到 raw.bin 文件中
- B. 可以使用命令 `hex2raw < hex.txt > raw.bin` 将 hex.txt 中的内容转换为字节序列并保存到 raw.bin 文件中
- C. 在得到文件 raw.bin 后，可以使用命令 `./ctarget < raw.bin` 将 raw.bin 作为输入传递给目标程序 ctarget
- D. 如果修改了 hex.txt 文件的内容，必须重新编译 hex2raw 工具才能正确转换新的内容

22.（2 分）下列关于 AttackLab 的描述中，哪个选项是正确的？

- A. AttackLab 的目标程序（target）往往包含缓冲区溢出漏洞，这个漏洞只存在于 getbuf 函数的栈帧中
- B. 在 AttackLab 中，我们尝试进行了空操作雪橇（nop sled）攻击
- C. ctarget, starget, rtarget 都没有使用地址空间布局随机化（ASLR）技术，因此可以直接注入可执行的攻击代码
- D. 因为存在金丝雀值保护机制，所以我们不能在栈上直接注入可执行的攻击代码，而必须使用 ROP 技术

23.（2 分）在 AttackLab 中，gdb 调试器用于调试目标程序，分析程序的运行状态和内存布局。假设现在需要调试一个名为 ctarget 的目标程序，下列哪个选项是正确的？

- A. 使用命令 `gdb ctarget` 启动 gdb 调试器后，想要调试的程序自动开始运行
- B. 如果我们想要以十六进制格式打印出寄存器 \$rdi 的值，可以使用命令 `x/x $rdi`
- C. 可以使用命令 `break getbuf` 或 `b getbuf` 在 getbuf 函数入口处设置断点
- D. 在 gdb 中，可以使用命令 `run > input.txt` 将文件 input.txt 的内容作为输入传递给目标程序

24. (2 分) 下面给出了 AttackLab 中 ctarget 目标程序的 getbuf 函数的汇编代码：  
在 call 指令执行完成后，getbuf 的栈帧大小为：

- A. 0x38 字节
- B. 0x3C 字节
- C. 0x40 字节
- D. 0x44 字节

0000000000401cb2 <getbuf>:

```
401cb2: f3 0f 1e fa      endbr64
401cb6: 48 83 ec 38      sub    $0x38,%rsp
401cba: 48 89 e7         mov    %rsp,%rdi
401cbd: e8 57 03 00 00   call   402019 <Gets>
401cc2: b8 01 00 00 00   mov    $0x1,%eax
401cc7: 48 83 c4 38      add    $0x38,%rsp
401ccb: c3              ret
```

25. (2 分) 在 AttackLab 的 phase3 中，我们需要通过缓冲区溢出来执行特定的函数 touch3。下面给出了 touch3 以及其调用的 hexmatch 的函数原型，以及 phase3 的一种题解。判断下面说法错误的是：

- A. 缓冲区的大小为 56 字节
- B. 在执行 movq &cookie, %rdi 指令时，寄存器 %rsp 的值是 0x5566bd88
- C. 当执行完 movq &cookie, %rdi 指令后，寄存器 %rdi 的值是 0x5566bda0
- D. 如果 cookie 的字符串表示末尾的 00 字节是为了填充对齐而添加的，hexmatch

函数在比较时会忽略该字节

```
/* Compare string to hex representation of unsigned value */
int hexmatch(unsigned val, char *sval)
{
    char cbuf[110];
    /* Make position of check string unpredictable */
    char *s = cbuf + random() % 100;
    sprintf(s, "%.8x", val);
    return strncmp(sval, s, 9) == 0;
}
```



```

void touch3(char *sval)
{
    vlevel = 3; /* Part of validation protocol */
    if (hexmatch(cookie, sval)) {
        printf("Touch3!: You called touch3(\"%s\")\n", sval);
        validate(3);
    } else {
        printf("Misfire: You called touch3(\"%s\")\n", sval);
        fail(3);
    }
    exit(0);
}

// phase3 的一种题解
/* padding with 56 bytes */
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
/* Address of exploit code movq &cookie, %rdi */
98 bd 66 55 00 00 00 00
/* Address of a ret in exploit code, used for %rsp alignment */
9f bd 66 55 00 00 00 00
/* Address of touch3 */
98 1e 40 00 00 00 00 00
/* movq &cookie, %rdi */
48 c7 c7 a0 bd 66 55
/* ret */
c3
/* String representation of cookie */
32 64 36 66 63 32 64 35 00

```

## 实验 L4 (10 分) 得分: \_\_\_\_\_

| 题号 | 26 | 27 | 28 | 29 | 30 |
|----|----|----|----|----|----|
| 解答 |    |    |    |    |    |

26. (2 分) 在 ArchLab Part B 中, 我们需要在 SEQ 架构的基础上实现 iopq 指令。该指令的定义如下: iopq V, rB: 计算 rB op V, 并将结果存入寄存器 rB。下面是 SEQ 架构中的部分 HCL-rs 定义, 其中哪些位置需要添加 IOPQ?

- A. (1) (2) (3)
- B. (1) (3) (5)
- C. (2) (3) (5)
- D. (2) (4) (5)

```

u8 srcA = [
    icode in { CMOVX, RMMOVQ, OPQ, PUSHQ } : ialign.rA; // (1)
    icode in { POPQ, RET } : RSP;
    true : RNONE;
];

u8 dstE = [
    icode in { CMOVX } && cnd : ialign.rB;
    icode in { IRMOVQ, OPQ } : ialign.rB; // (2)
    icode in { PUSHQ, POPQ, CALL, RET } : RSP;
    true : RNONE;
];

u64 aluA = [
    icode in { CMOVX, OPQ } : reg_read.valA; // (3)
    icode in { IRMOVQ, RMMOVQ, MRMVQ } : ialign.valC; // (4)
    icode in { CALL, PUSHQ } : NEG_8;
    icode in { RET, POPQ } : 8;
];

u8 alufun = [
    icode == OPQ : ifun; // (5)
    true : ADD;
];

```

27. (2分) 在 ArchLab Part B 中, 我们将架构从 SEQ 一步步改进到 pipe\_std 这个标准的五级流水线架构。下列说法中错误的一项是?
- A. 随着架构的一步步改进, 关键路径的长度逐渐变短, 时钟频率可以逐渐提高
  - B. 从 pipe\_s2 演化到 pipe\_s3a 的过程中引入了数据冒险, 可通过增加寄存器解决
  - C. pipe\_s4a 将架构中的执行阶段分离出来, 使内存访问和 ALU 计算可以并行执行
  - D. 标准的五级流水线 pipe\_std 具有分支预测, 可能发生分支预测错误
28. (2分) ArchLab Part C 中, 我们需要优化 ncopy.py 代码。下面两个代码片段截取自某两个同学的 ncopy.py, 它们均使用标准的五级流水线架构。下列关于这两段代码性能的说法, 正确的一项是?
- A. 代码 S 性能更优, 因为将访问内存的指令集中在了一起, 可以有效增加缓存命中率
  - B. 代码 S 性能更优, 因为将功能相似的指令如 iaddq 集中在一起, 便于流水线优化
  - C. 代码 T 性能更优, 因为将部分公共指令提前执行, 使每个循环中平均指令数量变少
  - D. 代码 T 性能更优, 因为将 iaddq 指令插入到了前面, 避免了加载/使用冒险的发生

| 代码 S                                                                                                                                                                                                   | 代码 T                                                                                                                                                                                                   |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> Loop:     mrmovq (%rdi), %r10     rmmovq %r10, (%rsi)     andq %r10, %r10     jle Skip     iaddq \$1, %rax Skip:     iaddq \$8, %rdi     iaddq \$8, %rsi     iaddq \$-1, %rdx     jg Loop </pre> | <pre> Loop:     mrmovq (%rdi), %r10     iaddq \$8, %rdi     rmmovq %r10, (%rsi)     iaddq \$8, %rsi     andq %r10, %r10     jle Skip     iaddq \$1, %rax Skip:     iaddq \$-1, %rdx     jg Loop </pre> |

29. (2分) 在 ArchLab Part C 中, 我们需要同时优化代码 ncopy.py 和架构 ncopy.rs, 评分标准基于  $c = cpe + 2 * ac$ 。关于这一部分的优化策略, 下列说法中正确的一项是?
- A. 应该优先优化 cpe, 因为 ac 的权重不大, 且 cpe 不依赖于架构的设计
  - B. 应该优先优化 ac, 因为这样可以提高时钟频率, 降低执行与调试的时间
  - C. cpe 和 ac 相互独立, 优化 cpe 不会影响 ac, 优化 ac 也不会影响 cpe
  - D. 在 ncopy.rs 中添加更多的流水线阶段可以降低 ac, 但可能使 cpe 增加

30. (2分) 在 ArchLab Part C 中, 某同学准备在标准的五级流水线架构 `pipe_std` 的基础上进一步优化。该同学输入命令 `./target/debug/ysim --arch pipe_std -I`, 得到了下面的结果。该同学进行下列哪一项优化, 可以在降低 `ac` 的同时, 不使 `cpe` 显著增加?
- A. 在架构中增加 `iopq` 指令, 提高部分指令的执行效率
  - B. 在架构中缩短 `e_cnd`, `d_bubble` 等信号的路径长度
  - C. 在架构中执行和访存阶段中间增加“设置条件码”阶段
  - D. 在架构中将设置条件码从执行阶段改到访存阶段进行

propagate order:

**lv.1:** w\_stall m\_icode d\_dstM m\_dstM prog\_stat mem\_write d\_srcB e\_dstM  
m\_stall m\_valE d\_dstE w\_dstM e\_ifun e\_stat e\_icode d\_stat f\_pc aluB  
mem\_read e\_valA m\_dstE alufun w\_bubble d\_icode w\_valE e\_stall aluA  
mem\_data f\_bubble d\_srcA mem\_addr d\_valC d\_ifun w\_dstE w\_valM prog\_term  
d\_stall f\_stall

**lv.2:** imem alu dmem reg\_file f\_align f\_icode f\_ifun e\_valE m\_valM  
m\_stat d\_rvalA d\_rvalB instr\_valid need\_regids need\_valC m\_bubble  
set\_cc f\_stat

**lv.3:** ialign pc\_inc reg\_cc f\_rA f\_rB f\_valC f\_valP cc f\_pred\_pc

**lv.4:** cond e\_cnd d\_bubble e\_bubble e\_dstE d\_valA d\_valB

dependency graph visualization is generated at:

`pipe_std_dependency_graph.html`

实验 L5 (10 分) 得分: \_\_\_\_\_

| 题号 | 31 | 32 | 33 | 34 | 35 |
|----|----|----|----|----|----|
| 解答 |    |    |    |    |    |

31. (2 分) 在 CacheLab 实验中, valgrind 工具的主要用途是什么?

- A. 作为参考缓存模拟器 (csim-ref) 来检查输出结果
- B. 编译 csim.c 和 trans.c 文件
- C. 生成程序运行时的内存访问轨迹 (trace files)
- D. 自动评估 Part B 矩阵转置的性能得分

32. (2 分) 在 CacheLab Part A (csim.c) 中, 模拟器需要解析轨迹文件中的内存访问操作。根据 writeup, 下列哪个操作类型应该被忽略?

- A. M (数据修改)
- B. L (数据加载)
- C. S (数据存储)
- D. I (指令加载)

33. (2 分) 根据 CacheLab Part A 的编程规则和提示, 当缓存模拟器 (csim.c) 解析到一行 M (数据修改) 操作时, 它应该如何处理?

- A. 将其视为单次内存访问, 最多导致一次缓存未命中 (miss) 和一次可能的淘汰 (eviction)
- B. 将其视为一次数据加载 (L) 和紧随其后的一次数据存储 (S)。这可能导致两次命中, 或者一次未命中和一次命中 (可能伴随淘汰)
- C. 忽略此操作, 因为实验只关心数据加载 (L) 和数据存储 (S) 操作
- D. 只将其视为一次数据存储 (S) 操作, 因为数据最终被修改了

34. (2 分) 针对 CacheLab Part B 中朴素的逐行扫描转置 (`for i... for j... B[j][i] = A[i][j]`) 性能不佳, 因为它在写入矩阵 B 时具有很差的空间局部性。Blocking (分块) 策略的核心思想是如何解决这个问题的?

A. 交换 i 和 j 循环的顺序改为逐列扫描, 这样 A 的访问变差, 但 B 的访问具有良好的空间局部性

B. 一次只处理一个对角线元素, 使用局部变量来存储它, 以避免冲突不命中, 然后再处理非对角线元素

C. 将大矩阵划分为适合装入缓存的、小的子矩阵 (块)。通过一次只对一个块进行转置, 可以最大化局部性, 即重复使用刚从 A 读入缓存的数据, 同时也改善了对 B 对应块写入的局部性

D. 利用 12 个局部变量, 一次性从 A 读取 12 个元素, 转置后, 再将这 12 个元素写入 B 的不同行中

35. (2 分) 在 CacheLab Part A 编写缓存模拟器 (`csim.c`) 时, 推荐使用哪个 C 语言函数来解析命令行参数 (如 `-s`, `-E`, `-b` 等)?

A. `scanf`

B. `getopt`

C. `mmap`

D. `fopen`

## 实验 L6 (10 分) 得分: \_\_\_\_\_

| 题号 | 36 | 37 | 38 | 39 | 40 |
|----|----|----|----|----|----|
| 解答 |    |    |    |    |    |

36. (2 分) 在 ShellLab 中, 我们实现了一个简单的 shell 程序 tsh。和真实的 Linux shell 相比, 下列哪一项说法是错误的?

A. tsh 中不支持变量, 也不支持搜索路径, 无法直接运行系统程序如 cat, 需要输入完整路径/bin/cat

B. tsh 中不支持管道 (|), 但支持 I/O 重定向 (<, >), 且可以在一条命令里同时进行输入、输出重定向

C. tsh 和真实的 shell 一样, 需要捕获 SIGINT 和 SIGTSTP 信号, 然后再将这些信号发送给前台进程组

D. tsh 和真实的 shell 一样, 支持通过 & 设置程序在后台执行, 也支持通过 fg 命令将后台任务转移到前台

37. (2 分) 在 ShellLab 中, 为了避免出现竞争 (race), 需要在特定的操作前屏蔽一些信号。下列关于屏蔽信号与解除屏蔽的时机, 说法错误的一项是?

A. sigint\_handler 中不需要屏蔽信号, 因为只需要读取全局数据结构 job\_list, 而无需对其进行修改

B. sigchld\_handler 中需要屏蔽信号, 因为需要修改全局数据结构 job\_list, 更新 job 的状态等相关信息

C. eval 中需要屏蔽信号, 因为需要修改全局数据结构 job\_list, 在创建新任务时添加新创建的 job 信息

D. fork 之后需要解除屏蔽信号, 解除的时机可以在执行 setpgid 之前, 也可以在执行 setpgid 之后

38. (2 分) 在 ShellLab 中, 我们需要实现一系列内建命令。下列关于内建命令的说法正确的是?

- A. jobs 命令中我们需要列出所有任务, 且需要先屏蔽所有相关信号, 避免竞争
- B. bg 命令中我们需要将当前运行在前台的任务变为后台运行, 且不使任务停止
- C. kill 命令中我们需要向指定的进程或进程组发送 SIGKILL 信号以将其终止
- D. nohup 命令中我们需要在创建子进程后阻塞 SIGHUP 信号或设置忽略该信号

39. (2 分) 下面是 ShellLab 中 sigchild\_handler 的一个正确实现中的部分代码(省略了部分检查与输出)。则代码中 (1) 和 (2) 应该分别为?

- A. (1) WIFSTOPPED (2) WIFCONTINUED
- B. (1) WIFSTOPPED (2) WIFSIGNALED
- C. (1) WIFEXITED (2) WIFCONTINUED
- D. (1) WIFEXITED (2) WIFSIGNALED

```
while ((child_pid = waitpid(-1, &status, WNOHANG | WUNTRACED |  
WCONTINUED)) > 0) {  
    if (____(1)____(status)) {  
        struct job_t *j = getjobpid(job_list, child_pid);  
        j->state = ST;  
    }  
    else if (____(2)____(status)) {  
        struct job_t *j = getjobpid(job_list, child_pid);  
        if (!(j->state == FG)) {  
            j->state = BG;  
        }  
    }  
    // ...  
}
```



40. (2 分) 在 ShellLab 中, 我们在 sigchld\_handler 中使用 waitpid 来回收子进程, 同时我们定义了 waitfg 函数用来等待特定 pid 的前台任务结束。下面 waitfg 函数的实现, 正确的一项是?

---

A.       void waitfg(pid\_t pid) {  
          int timeout = 0;  
          struct job\_t \*j = getjobpid(job\_list, pid);  
          while (j->pid == pid && j->state == FG) {  
              if (++timeout > MAX\_TIME) {  
                  break;  
              }  
          }  
      }

---

B.       void waitfg(pid\_t pid) {  
          struct job\_t \*j = getjobpid(job\_list, pid);  
          while (j->pid == pid && j->state == FG) {  
              sleep(1);  
          }  
      }

---

C.       sigset\_t oldmask;  
          void waitfg(pid\_t pid) {  
              struct job\_t \*j = getjobpid(job\_list, pid);  
              while (j->pid == pid && j->state == FG) {  
                  sigsuspend(&oldmask);  
              }  
          }

---

D.       int status;  
          void waitfg(pid\_t pid) {  
              struct job\_t \*j = getjobpid(job\_list, pid);  
              while (j->pid == pid && j->state == FG) {  
                  waitpid(pid, &status, 0);  
              }  
          }

---

实验 L7 (10 分) 得分: \_\_\_\_\_

| 题号 | 41 | 42 | 43 | 44 | 45 |
|----|----|----|----|----|----|
| 解答 |    |    |    |    |    |

41. (2 分) MallocLab 实现的分配器需保证返回的指针满足什么样的对齐要求?

- A. 4 字节对齐
- B. 8 字节对齐
- C. 16 字节对齐
- D. 根据请求大小动态对齐

42. (2 分) 在 MallocLab 中关于系统调用和库函数的使用, 以下说法正确的是?

- A. 可以使用 libc 的 malloc
- B. 可以使用 sbrk 系统调用来扩展堆
- C. 必须通过调用 memlib.c 中的 mem\_sbrk 函数来扩展堆
- D. 分配大空间时, 可以使用 mmap 等内存管理函数

43. (2 分) 以下关于各函数行为的描述中, 哪一项是正确的?

- A. free(NULL) 是非法的, 会导致程序崩溃或未定义行为
- B. 当 realloc 为了扩容而将数据移动到新地址时, 旧块会自动被释放, 用户不需要再调用其他函数释放该块
- C. calloc 与 malloc 区别仅在于参数形式不同, 两个函数分配的内存区域都是未初始化的
- D. realloc(ptr, 0) (其中 ptr 不为 NULL) 的行为是将该块的大小标记为 0, 但保留该块的已分配状态, 并返回原指针

44. (2 分) 在设计分配器时, 需要选择合适的策略来在空闲链表中查找空闲块。以下关于查找策略以及空间利用率和吞吐量的描述, 哪一项是正确的?

A. 获得更高的空间利用率往往需要对空闲块进行更彻底 (因此也更慢) 的扫描, 以选出最适合当前需求的块

B. 为了追求极致的吞吐量, 总是应该使用最佳适配 (best fit) 策略, 因为它能最快定位到大小刚好合适的块

C. 高空间利用率和运行速度之间不存在冲突, 好的算法可以同时最大化这两个指标

D. 对于内存较大的机器, 在设计分配器时应当更关注空间利用率, 而不是吞吐量

45. (2 分) 虽然实验环境是 64 位的, 但 writeup 指出了堆大小有一个特殊限制, 并且该限制为我们提供了一种优化思路, 这个限制是?

A. 堆的大小不会超过  $2^{32}$  字节

B. 堆的大小不会超过  $2^{64}$  字节

C. 堆大小受限于物理内存大小

D. 堆中的数据不会被交换到外部存储中

实验 L8 (10 分) 得分: \_\_\_\_\_

| 题号 | 46 | 47 | 48 | 49 | 50 |
|----|----|----|----|----|----|
| 解答 |    |    |    |    |    |

46. (2 分) IPv4 和 IPv6 的地址长度是多少位?
- A. 32 位, 64 位
  - B. 32 位, 128 位
  - C. 64 位, 128 位
  - D. 64 位, 256 位
47. (2 分) 关于 TCP 和 UDP 协议的区别, 以下说法正确的是:
- A. TCP 是面向连接的, UDP 是无连接的
  - B. TCP 不保证可靠性, UDP 保证可靠性
  - C. TCP 传输速度比 UDP 快
  - D. TCP 支持广播, UDP 不支持广播
48. (2 分) IP 地址 192.168.1.100/24 所在的子网中, 可用的主机地址数量是:
- A. 24
  - B. 254
  - C. 256
  - D.  $2^{24}$
49. (2 分) 用个人笔记本连接北大无线网, 可能获得的 IP 地址是 10.7.74.125, 这个 IP 地址属于:
- A. A 类地址 (0 开头, 加 7 位 Net ID)
  - B. B 类地址 (10 开头, 加 14 位 Net ID)
  - C. C 类地址 (110 开头, 加 21 位 Net ID)
  - D. 私有地址 (Unrouted IP addresses)
50. (2 分) 以下说法正确的是:
- A. 建立套接字连接的客户端和服务端端的 IP 地址不能相同
  - B. IP 协议提供了可靠的主机与主机之间的数据包传输能力
  - C. 一个域名可以映射到多个 IP, 也可以不映射到任何 IP
  - D. 为了防止内存泄漏, 使用 `getaddrinfo` 函数获得的 `addrinfo` 链表只能调用 `freeaddrinfo` 函数释放